Data-Intensive AI Systems: Building ML Infrastructure from First Principles

Student number: s242796

1. Introduction

Modern machine learning systems depend on data infrastructure that can store features, update them from new events, and recover from failures. The goal of MLStore-Lite is to build a small educational prototype of such a system. The project is inspired by the distributed data-system concepts from *Designing Data-Intensive Applications*, but keeps the implementation local and readable.

The project does not try to compete with production systems such as RocksDB, Cassandra, Kafka, Spark, Flink, or managed ML platforms. I do not claim deep practical knowledge of all of these frameworks. Instead, I use them as reference points for understanding the architecture of data-intensive systems. The goal is to build small versions of their central ideas so that the internal design becomes easier to inspect: write-ahead logging, sorted-string tables, replication, partitioning, batch processing, stream processing, observability, online model inference, sequential recommendation, metadata, lineage, and data quality can all be studied in one code base.

The final system can run a small feature workflow end to end. Raw historical events are converted into batch features, new events are appended to a stream log, stream consumers update windowed features, and all feature values are stored in a sharded replicated key-value store. The final extensions serve those features to a small model inference layer, train a tiny sequence-based recommender, and log predictions for inspection. The full local pipeline can be reproduced with one terminal command: `python -m mlstore_lite.experiments.final_demo`.

This project is also personal learning work. Coming from a mathematics background and having worked with code for a bit less than two years, I wanted to go deeper into software architecture rather than only use high-level libraries. DDIA was useful because it explains why systems are shaped the way they are: how data is stored, copied, partitioned, processed, and observed. In future iterations of this repository, I would like to revisit the project with the newer DDIA edition and add small implementations inspired by *System Design Interview*.

2. Reading Scope and Project Method

The reading scope followed the course proposal and focused on selected parts of *Designing Data-Intensive Applications* rather than the entire book. The second edition of the book was released in 2026 by Martin Kleppmann and Chris Riccomini. A future direction for this repository is to revisit the project with the newer edition in more depth, especially the parts on cloud-native systems, managed platforms, and operational tradeoffs.

The main DDIA reading and implementation mapping was:

Milestone	Main DDIA connection	What was implemented
Week 1: Storage engine	Chapter 3: Storage and Retrieval	Write-ahead log, memtable, SSTable-like files, compaction, and crash recovery
Week 2: Replication	Chapter 5: Replication	Leader-follower replication, synchronous/asynchronous writes, replication lag, and manual failover
Week 3: Sharding / partitioning	Chapter 6: Partitioning	Hash-based partitioning, consistent hashing, virtual nodes, request routing, and rebalancing
Week 4: Batch processing	Chapter 10: Batch Processing	Local MapReduce-style `map -> shuffle -> reduce` engine and batch feature computation
Weeks 5-6: Stream processing	Chapter 11: Stream Processing	Append-only event log, producers, consumers, offsets, tumbling windows, and stream feature updates

Milestone	Main DDIA connection	What was implemented
Week 7: Integration	DDIA system-design theme: composing storage and processing systems	One `MLStoreLiteSystem` that wires storage, replication, sharding, batch, and stream layers together
Week 8: Evaluation and observability	DDIA operational theme: understanding tradeoffs and system behavior	Structured logs, timing measurements, JSON-lines experiment records, and comparison with production tools
Week 9: Online feature serving and inference	Extension from data systems into ML infrastructure	Feature serving, deterministic model inference, confidence/warning output, and prediction logging
Week 10: Scaling and cloud design	DDIA partitioning/replication/processing tradeoffs revisited	Workload scaling experiment, shard hotspot experiment, and cloud architecture design sketch
Week 11: Sequential recommender	Extension from feature serving into sequence-based ML	Ordered user histories, token vocabulary, tiny attention model, prediction audit, and model card
Week 12: Metadata, lineage, and data quality	MLOps extension for inspection and traceability	Feature registry, event validation, quality reports, and prediction lineage

More specifically, the implementation was guided by the following DDIA sections and ideas:

Project part	DDIA section or subsection theme	How it appears in MLStore-Lite
Write path and recovery	Chapter 3: log-structured storage, write-ahead logs, and storage-engine internals	Writes are appended to a WAL before being stored in memory, so a node can recover after restart
MemTable and SSTables	Chapter 3: SSTables and LSM-trees	Recent writes live in a memtable and are flushed into immutable sorted files
Compaction	Chapter 3: merging and compaction in log-structured storage	Multiple SSTable-like files are merged so newer values replace older values
Leader-follower replication	Chapter 5: leaders and followers	A leader accepts writes and forwards them to follower replicas
Sync vs async replication	Chapter 5: synchronous versus asynchronous replication	The cluster supports both synchronous and asynchronous replication modes
Replication lag	Chapter 5: problems with replication lag	Async followers may temporarily be behind the leader
Failover	Chapter 5: handling node outages and failover	The project implements manual follower promotion, not automatic consensus
Partitioning by hash	Chapter 6: partitioning key-value data by hash of key	Feature keys are routed to shards using a hash ring
Virtual nodes	Chapter 6: rebalancing and partition distribution	Each shard appears multiple times on the ring to improve distribution
Hotspots	Chapter 6: skewed workloads and relieving hot spots	Week 10 compares balanced and hotspot workloads and records request pressure
Request routing	Chapter 6: request routing to partitions	`ShardedCluster` decides which shard owns each key
Batch dataflow	Chapter 10: MapReduce-style batch processing	The batch engine implements `map -> shuffle -> reduce` locally

Project part	DDIA section or subsection theme	How it appears in MLStore-Lite
Derived data	Chapter 10: outputs of batch workflows and derived views	Raw events are transformed into stored feature values
Event logs	Chapter 11: transmitting event streams and log-based messaging	Producers append events to a local event log
Consumers and offsets	Chapter 11: consumers, offsets, and partitioned logs	Consumers track how far they have read using an offset store
Windowed stream processing	Chapter 11: stream processing and reasoning about time	Events are grouped into tumbling windows before updating features
Observability and evaluation	DDIA's recurring emphasis on tradeoffs, operational behavior, and failure modes	Week 8 records timings, JSON-lines experiment results, and limitations
Cloud design	DDIA's broader discussion of data systems as composed services	Week 10 maps the local layers to possible cloud services such as Kafka, Spark, Flink, Cassandra, and model serving
Sequential recommendation	Extension beyond DDIA into ML systems using event histories	Week 11 uses ordered events from the pipeline as model input instead of only feature counts
Metadata and lineage	MLOps inspection theme	Week 12 explains feature meanings, checks event quality, and records prediction traces

Several DDIA topics were intentionally not implemented deeply. Transactions, consensus, automatic leader election, distributed commit protocols, real network partitions, and production-grade distributed recovery are discussed only as limitations or future work. This boundary was important because the project was meant to make the main architecture understandable, not to reproduce a full database, stream processor, or managed ML platform.

The implementation was AI-assisted. The code and documentation were developed with Codex 5.5, Claude Sonnet 4.7, and Claude Opus 4.7 as programming and writing assistants. I also experimented in some parts with local LLM models, including Qwen models around 35B parameters. The AI tools were used for iteration, explanation, implementation support, and debugging, while the project scope, interpretation, and final report decisions remained part of the learning process.

3. System Overview

MLStore-Lite is organized as a layered system. Each layer has a narrow responsibility and builds on the layer below it:

```
Storage engine
  -> Replication
  -> Sharding / partitioning
  -> Batch processing
  -> Stream processing
  -> Integration
  -> Evaluation and observability
  -> Online feature serving and model inference
  -> Sequential recommendation
  -> Metadata, lineage, and data quality
```

This structure is useful because it separates concerns. The storage engine only needs to know how one node persists key-value data. The replication layer only needs to know how several nodes keep copies of the same data. The sharding layer only decides which replicated group owns a key. Batch and stream processing then use the sharded store without needing to understand the lower-level storage details. The AI layer is the final consumer of the stored features.

4. Storage Engine

The storage engine answers the question:

`How does one node store data durably?`

It consists of:

- a write-ahead log
- an in-memory table
- immutable SSTable files
- compaction

When a key is written, the value is first appended to the write-ahead log. This means the operation can be recovered even if the process crashes before the in-memory data is flushed to disk. The in-memory table keeps recent writes fast. When it grows large enough, it is flushed into an SSTable file. SSTables are immutable sorted files, which makes reads and merges more predictable. Compaction later combines SSTables and removes overwritten values.

This is a simplified log-structured storage engine. The same broad idea appears in systems such as LevelDB and RocksDB, although production engines add many optimizations such as bloom filters, compression, concurrency control, and more advanced compaction strategies.

5. Replication

The replication layer answers the question:

`How can the system keep multiple copies of the same data?`

MLStore-Lite uses a leader-follower model. Each replicated group contains one leader and two followers, giving a replication factor of three.

Writes go to the leader. The leader stores the write locally and forwards it to followers. Reads are served through the current leader in the implemented interface. The project also supports manual failover, where a follower can be promoted to leader after a failure experiment.

Replication improves fault tolerance because one physical copy is no longer the only place where data exists. In this prototype the replicas are local objects and directories, not separate networked machines. This keeps the focus on the data-system concept rather than on networking.

6. Sharding and Partitioning

Replication creates copies of the same data. Sharding divides different keys across different replicated groups.

The sharding layer answers:

`Which shard owns this key?`

MLStore-Lite uses consistent hashing for routing. A key is hashed onto a ring, and the next shard on the ring owns that key. This means keys can be distributed without manually assigning ranges such as `users 1-1000 go to shard A`. When a shard is added, only part of the key space needs to move.

In the implemented topology, each shard is itself a replicated group with replication factor three. Therefore a shard is not just one file or one Python object. It is a logical partition of the key space, backed by multiple replica nodes.

7. Batch Processing

The batch layer answers the question:

`How can raw historical records be turned into useful derived data?`

In MLStore-Lite, the derived data are machine-learning-style features.

The batch engine follows a small MapReduce pattern:

`map -> shuffle -> reduce`

The map step reads one input event and emits intermediate key-value pairs. The shuffle step groups intermediate values by feature key. The reduce step combines the grouped values into final feature values. For example, several click events for the same user can become a single click-count feature.

The implemented batch feature job computes:

- per-user event count
- per-user click count
- per-user purchase count
- per-user total purchase amount

The results are written back into the sharded replicated store using keys such as:

`feature:user:42:click_count`

This mirrors the role of batch systems such as MapReduce and Spark, but at a small local scale.

8. Stream Processing and Integration

Batch processing is useful for historical data, but many ML systems also need fresh features from recent events. The stream layer answers:

`How can new events update derived data incrementally?`

MLStore-Lite implements:

- a small append-only event log
- producers
- consumers
- an offset store
- tumbling windows
- a stream feature processor

Producers append events to the log. Consumers read from the log and track their position using offsets. The processor groups events into fixed-size windows and writes updated features into the same sharded replicated store used by the batch layer.

This is similar in spirit to Kafka and stream processors such as Flink or Kafka Streams. Kafka provides the durable distributed event log, while Flink and Kafka Streams provide continuous computations over event streams. MLStore-Lite keeps a small local version of the same idea: new events are appended, consumed, grouped, and converted into feature updates.

The integration layer connects all previous components into one object:

`MLStoreLiteSystem`

This object creates the sharded replicated store, the batch feature job, the event log, producer, consumer, offset store, and stream processor. Its purpose is not to hide the architecture, but to make the whole prototype runnable without manually wiring every class each time.

The integrated system supports this workflow:

1. Compute batch features from historical events.
2. Produce new events into the stream log.
3. Process stream events into windowed features.
4. Read features from the sharded replicated store.
5. Inspect shard distribution and replica status.

This gives the project an end-to-end ML infrastructure story: raw events become features, and the features are stored in a distributed-system-inspired storage backend.

9. Evaluation and Observability

The evaluation layer adds local observability. In this project, observability means:

- structured logs
- timing measurements
- reproducible experiment records

The goal is not to build a monitoring platform. The goal is to make the prototype inspectable.

The Week 8 evaluation script builds the integrated system and measures:

- batch feature computation time
- stream event production time
- stream processing time
- feature read latency
- key distribution across shards

Each measurement is written to a JSON-lines file with:

- experiment name
- metric
- value
- unit
- timestamp
- parameters

The parameters include the number of events, number of shards, and replication factor. This is important for reproducibility: a runtime number is more meaningful when the conditions that produced it are also recorded.

The evaluation script also emits structured JSON logs. A structured log is more useful than plain text because it can be parsed by other tools. For example, the system can log that a measurement was recorded together with the operation name, elapsed time, and output count.

10. Online Feature Serving and Model Inference

The final extension adds a small online inference layer. This layer uses the features computed by the batch and stream processors and serves them to a deterministic purchase-intent model.

The final prediction pipeline is:

```
events
  -> batch features
  -> stream/window features
  -> sharded replicated feature store
  -> online feature server
  -> purchase intent model
  -> prediction log
```

The feature server reads expected user features from MLStore-Lite, including historical batch features and recent windowed stream features. It also records whether expected inputs are missing.

The model uses the feature values to produce:

- purchase probability
- confidence
- label
- warnings

Example prediction:


```
{
  "user_id": "0",
  "model_version": "purchase-intent-v1",
  "purchase_probability": 0.9953,
  "confidence": 1.0,
  "label": "likely_to_purchase",
  "warnings": []
}
```


If features are missing, the model lowers confidence and records warnings:

```
{
  "user_id": "999",
  "purchase_probability": 0.0989,
  "confidence": 0.32,
  "label": "uncertain",
  "warnings": [
    "missing_event_count",
    "missing_click_count",
    "missing_purchase_count",
    "missing_total_purchase_amount",
    "no_recent_stream_activity"
  ]
}
```


This is intentionally not a large model-training system. The purpose is to show the role of the data infrastructure in model serving:

`features must be computed, stored, served, checked, and logged.`

Predictions are written to a JSON-lines log with the user id, model version, input features, prediction probability, confidence, label, and warnings. This connects the AI extension back to MLOps: model outputs should be traceable and debuggable, not just returned and forgotten.

11. Sequential Recommender

The final AI extension adds a small sequential recommender. The earlier model uses feature counts. The recommender also uses the order of user events.

For example, these two histories can have similar counts but different meaning:

```
view laptop -> view laptop -> add_to_cart laptop
add_to_cart laptop -> view laptop -> view laptop
```

A sequence model can use the order. This is closer to how recommendation systems reason about user behavior.

The implemented model is a small Transformer-style model rather than a large deep-learning system. Events and items become tokens, tokens become numeric IDs, numeric IDs become embeddings, and attention weighs the recent sequence. The model then outputs a purchase probability, confidence, label, and the tokens that received the highest attention.

The training script can read the RetailRocket ecommerce dataset if it is placed locally under:

`data/raw/retailrocket/events.csv`

If the file is missing, the script uses a small built-in sample. This keeps the repository easy to run while still allowing a larger dataset later.

The recommender connects back to the rest of the system:

```
events
  -> user histories
  -> tiny attention recommender
  -> prediction log
  -> sharded store prediction keys
```

This makes the AI layer more realistic because it consumes ordered behavior instead of only manually chosen feature counts.

12. Metadata, Lineage, and Data Quality

The final polish layer adds three inspection tools.

The feature registry explains what stored feature keys mean. The storage engine only sees keys and values, but a reader needs to know that `feature:user:42:click_count` means the number of historical click-like events for a user. The registry records the feature name, key pattern, source layer, value type, description, and models that use it.

The quality layer validates raw events before they are processed. It checks simple rules such as required user ids, known event types, valid timestamps, and non-negative amounts. Invalid events are written into a quality report and skipped in demos.

The lineage layer records which inputs were used for a prediction. For the feature-count model, a lineage record stores the input feature keys, missing features, model version, and output prediction keys. For the sequential recommender, it records the number of user events and tokens used for the prediction.

This layer is intentionally small. It does not add an enterprise metadata service. It adds enough traceability to ask:

```
What does this feature mean?
Was the input event valid?
Which data did this prediction use?
```

13. Comparison With Production Tools

MLStore-Lite is a teaching prototype, so the comparison with production systems is conceptual rather than a performance benchmark.

MLStore-Lite layer	Production reference	Shared idea
Storage engine	RocksDB / LevelDB	WAL, memtable, SSTables, compaction
Sharding and replication	Cassandra	Partitioned key space, replicated data
Event log	Kafka	Append-only events, producers, consumers, offsets
Batch processing	Spark / MapReduce	Map, shuffle, reduce over finite data
Stream processing	Flink / Kafka Streams	Continuous event processing and windowed state
Integrated workflow	Databricks / managed platforms	End-to-end data and ML infrastructure
Inference extension	Feature stores / model serving systems	Online feature retrieval and prediction logging
Sequential recommender	Recommendation systems	Ordered user behavior, attention, and purchase prediction
Metadata, lineage, and quality	Feature catalogs / ML observability tools	Feature definitions, data checks, and prediction traceability

The main difference is scale and operational complexity. Production tools run across real machines, handle concurrent users, recover from many failure modes, and optimize for performance. MLStore-Lite runs locally and focuses on making the core ideas understandable.

14. Limitations

The most important limitation is that the project is local. Nodes are Python objects and directories, not independent networked processes.

The project does not include:

- real network transport
- automatic leader election
- background repair
- distributed query execution
- production-grade monitoring
- model registry

- HTTP model serving API
- a deep implementation of DDIA topics such as transactions and consensus
- production deep-learning training infrastructure
- enterprise-grade feature catalog, lineage UI, or data quality platform

The evaluation is also intentionally small. The measurements are useful for checking that the system runs and for discussing relative behavior, but they are not benchmarks against real systems. The synthetic workloads are small enough to keep the project readable and runnable on a laptop.

As a final extension, I added local scaling and hotspot experiments. These experiments increase event counts and compare balanced versus skewed workloads. They still run locally, but they make scaling questions more concrete: runtime grows with input size, and hash partitioning does not automatically remove all request skew. A cloud version would keep the same conceptual flow but replace local files and Python objects with services such as event streaming, distributed batch/stream processing, distributed feature storage, model serving, and central observability.

These limitations are acceptable because the goal is educational. The project prioritizes clarity of architecture over production completeness.

15. Conclusion

MLStore-Lite implements a compact data infrastructure prototype for ML-style features. The project starts from a single-node storage engine and gradually adds replication, sharding, batch processing, stream processing, integration, observability, online model inference, a small sequential recommender, and a metadata/lineage/quality layer.

The final result shows how raw events can become derived feature values, and how those values can be stored in a sharded replicated backend, served to a model, and logged as predictions.

The project also shows why evaluation and observability matter. Once a system has several layers, it needs logs, measurements, and reproducible records to be understandable.

The final architecture can be summarized as:

```
events -> quality checks -> features -> feature store -> inference -> lineage -> prediction logs
```